

Relatório Técnico — Sistema IoT para Monitoramento de Cápsula Espacial

Global	Solution	2026	(1º	semestre)	—	FIAP
Turma:			1			CCPX
Plataforma:						Tinkercad
Projeto:	Mission Control AI — Cápsula Espacial IoT					

Integrantes

Gabriel Barbosa Furin — RM: 572941

Lucas Kiodi Moraca — RM: 571004

Renan Fracalossi Mano da Silva — RM: 569610

Introdução

A exploração espacial é uma das atividades humanas que mais exige confiabilidade dos sistemas tecnológicos. Em missões tripuladas ou não tripuladas, o monitoramento contínuo das condições físicas internas de uma capsula espacial é essencial para garantir a segurança dos astronautas, a integridade dos equipamentos e o sucesso operacional da missão.

Variáveis como temperatura interna, nível de luminosidade e intensidade de vibração estrutural são indicadores críticos do estado da capsula. Variações fora dos limites projetados podem sinalizar desde falhas no sistema de controle térmico até impactos mecânicos ou turbulências que comprometam a estrutura do módulo.

Nesse contexto, sistemas baseados em Internet das Coisas (IoT) e microcontroladores embarcados oferecem uma solução viável para coleta, processamento e exibição de dados em tempo real, com custo reduzido e alta flexibilidade de configuração. Este projeto propõe o desenvolvimento de um sistema IoT simulado no Tinkercad, utilizando um Arduino Uno como plataforma de controle, sensores de temperatura e luminosidade, um potenciômetro para simular vibração e um display LCD 16x2 para exibição dos dados monitorados.

Objetivo do Projeto

O objetivo deste projeto é desenvolver um sistema embarcado baseado em IoT para monitorar variáveis físicas essenciais para a operação segura de um módulo espacial durante uma missão simulada. O sistema deve:

- Coletar leituras continuamente dos sensores de temperatura, luminosidade e vibração;
- Processar os valores obtidos e compará-los com limites predefinidos (thresholds);
- Exibir os dados e o status do sistema em um display LCD 16x2;
- Acionar um LED de alerta sempre que qualquer variável monitorada ultrapassar seu limite crítico;
- Alternar automaticamente entre duas telas de informação no display, sem bloquear o ciclo de leitura.

Componentes Utilizados

A tabela a seguir lista todos os componentes empregados na simulação e descreve brevemente a função de cada um no sistema.

Componente	Quantidade	Função no Sistema
Arduino Uno	1	Microcontrolador principal; executa o firmware de aquisição e controle.
Display LCD 16x2	1	Exibe os valores lidos e o status de alerta em tempo real.
Potenciômetro (contraste)	1	Ajusta o contraste do display LCD.
Sensor TMP36	1	Mede a temperatura interna da capsula (entrada analógica A0).
LDR (foto resistor)	1	Captura o nível de luminosidade ambiente (entrada analógica A1).
Resistor 10 kohm	1	Forma o divisor de tensão com o LDR para leitura analógica.
Potenciômetro (vibração)	1	Simula a intensidade de vibração/impacto (entrada analógica A2).
LED vermelho	1	Indicador visual de alerta geral; aciona no pino digital 8.
Resistor ~220 ohms	1	Limita a corrente no LED de alerta.

Desenvolvimento do Circuito

O circuito foi montado integralmente na plataforma de simulação Tinkercad. A seguir, descrevem-se as ligações principais de cada subsistema.

1.1 Sensor de Temperatura — TMP36 (A0)

O sensor TMP36 possui três terminais: alimentação (5 V), saída analógica e terra (GND). Sua saída fornece uma tensão proporcional a temperatura, segundo a equação $T(\text{dec.}) = (\text{Volt} - 0,5) \times 100$. O terminal de saída é conectado diretamente ao pino analógico A0 do Arduino.

1.2 Sensor de Luminosidade — LDR com Divisor de Tensão (A1)

O LDR é conectado em série com um resistor fixo de 10 kohm, formando um divisor de tensão entre 5 V e GND. O nó central do divisor — ponto de junção entre o LDR e o resistor — é ligado ao pino analógico A1. Quanto menor a luminosidade ambiente, maior a resistência do LDR e, portanto, menor a tensão lida no pino A1.

1.3 Simulação de Vibração — Potenciômetro (A2)

A vibração não pode ser diretamente simulada com os componentes eletrônicos disponíveis no Tinkercad. Como alternativa equivalente para fins de validação da lógica de aquisição, processamento e alerta, um potenciômetro foi conectado ao pino analógico A2. O cursor do potenciômetro fornece uma tensão variável entre 0 V e 5 V, mapeada para valores de 0 a 1023 pelo conversor AD do Arduino. No firmware, o valor bruto do potenciômetro é invertido ($\text{vibNivel} = 1023 - \text{rawVib}$) de forma que a posição máxima do cursor represente a intensidade máxima de vibração.

1.4 Display LCD 16x2 — Modo 4 Bits

O display é controlado no modo de 4 bits, utilizando os seguintes pinos do Arduino: RS no pino 12, Enable no pino 11, D4 no pino 5, D5 no pino 4, D6 no pino 3 e D7 no pino 2. Um potenciômetro separado é conectado ao pino VO do display para ajuste de contraste, com os terminais de alimentação ligados a 5 V e GND.

1.5 LED de Alerta (Pino Digital 8)

Um LED vermelho é conectado ao pino digital 8 do Arduino, em série com um resistor de aproximadamente 220 ohm para limitação de corrente. O LED é acionado em nível alto (HIGH) sempre que ao menos uma das variáveis monitoradas ultrapassar seu respectivo limite crítico.

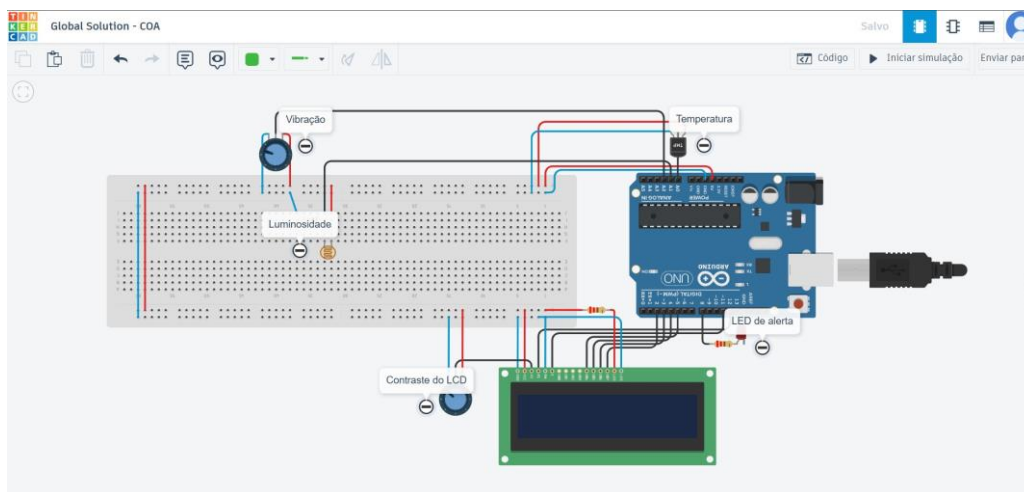


Figura 1 — Circuito completo montado no Tinkercad, exibindo todos os componentes e suas conexões.

Código Desenvolvido

O firmware foi desenvolvido em linguagem C/C++ para o Arduino, utilizando a biblioteca LiquidCrystal para comunicação com o display LCD. A seguir, são apresentados os trechos principais do código com as respectivas explicações.

1.6 Declaração de Constantes e Pinos

```
#include <LiquidCrystal.h>

// LCD: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

const int PIN_TEMP = A0; // TMP36 const int
PIN_LUZ = A1; // LDR (divisor) const int
PIN_VIB = A2; // Pot (simula vibracao) const
int PIN_LED_ALERTA = 8;

const float TEMP_ALERTA_C =
30.0; const int LUZ_ALERTA =
150; const int VIB_ALERTA = 700;
```

Neste bloco, os pinos analógicos e digital são mapeados para constantes com nomes descritivos, e os limites de alerta de cada variável são declarados como constantes para facilitar ajustes futuros de calibração sem alterar a lógica do programa.

1.7 Conversão de Temperatura — TMP36

```
float lerTemperaturaC_TMP36(int leituraAnalogica)
{ float tensao = leituraAnalogica * (5.0 / 1023.0);
return (tensao - 0.5) * 100.0;
}
```

A função converte a leitura bruta do ADC (0 a 1023) em graus Celsius, conforme a equação característica do sensor TMP36. Primeiro, a leitura é convertida para tensão em volts; em seguida, aplica-se o offset de 0,5 V e o fator de escala de 100 degC/V.

1.8 Inicialização — setup()

```
void setup() {
pinMode(PIN_LED_ALERTA, OUTPUT);
digitalWrite(PIN_LED_ALERTA, LOW); lcd.begin(16,
2); lcd.clear(); lcd.setCursor(0, 0);
lcd.print("Mission Control"); lcd.setCursor(0, 1);
lcd.print("Iniciando..."); delay(1200);
Serial.begin(9600);
}
```

Na inicialização, o LED é configurado como saída e colocado em estado baixo. O LCD é inicializado com 16 colunas e 2 linhas, exibindo uma mensagem de boas-vindas por 1,2 s antes do início das leituras. A comunicação serial é habilitada para depuração.

1.9 Lógica de Alertas e Acionamento do LED

```
bool alertaTemp = tempC >= TEMP_ALERTA_C;
bool alertaLuz = rawLuz <= LUZ_ALERTA;
bool alertaVib = vibNivel >= VIB_ALERTA;

bool alertaGeral = alertaTemp || alertaLuz || alertaVib;
digitalWrite(PIN_LED_ALERTA, alertaGeral ? HIGH : LOW);
```

Cada variável é avaliada individualmente contra seu limite crítico. O alerta geral é ativado por condição OR: basta uma variável fora do limite para acionar o LED e indicar estado de alerta no display. Para a vibração, o valor bruto é invertido antes da comparação, de modo que valores altos de vibNivel correspondam a maior intensidade.

1.10 Alternância de Telas no LCD

```
const unsigned long SCREEN_INTERVAL_MS = 3000;

if (now - lastScreenChange >= SCREEN_INTERVAL_MS)
{ screenIndex = (screenIndex + 1) % 2;
  lastScreenChange = now; }
```

A alternância de telas é implementada com `millis()`, evitando o uso de `delay()` e garantindo que o ciclo de aquisição não seja interrompido. A cada 3 segundos, o índice de tela muda entre 0 (valores + status geral) e 1 (diagnostico por variável).

1.11 Conteúdo das Telas do LCD

```
// TELA 0 – valores numericos e status geral
// Linha 0: "T:26.5C L:320"
// Linha 1: "V:85 G:OK"

// TELA 1 – diagnostico por variavel
// Linha 0: "T:OK L:ALRT"
// Linha 1: "V:OK G:ALRT"
```

A Tela 0 exibe os valores numéricos de temperatura (em graus Celsius, uma casa decimal), luminosidade (valor ADC bruto) e nível de vibração, além do status geral abreviado. A Tela 1 exibe o status individual de cada variável (OK ou ALRT) e o status geral, permitindo identificação rápida da variável que gerou o alerta.

Resultados Obtidos

O sistema foi validado por meio de testes funcionais realizados na plataforma Tinkercad. Durante os testes, os sensores e o potenciômetro foram ajustados manualmente para verificar o comportamento do sistema em diferentes cenários operacionais.

O circuito completo, com todos os componentes devidamente conectados, pode ser visualizado na Figura 1, apresentada na seção anterior.

Durante a operação em condições normais — temperatura abaixo de 30 degC, luminosidade acima de 150 e vibração abaixo de 700 —, o display exibe os valores numéricos na Tela 0 e o status 'G:OK' em ambas as telas. O LED permanece apagado. Este comportamento é ilustrado na Figura 2.

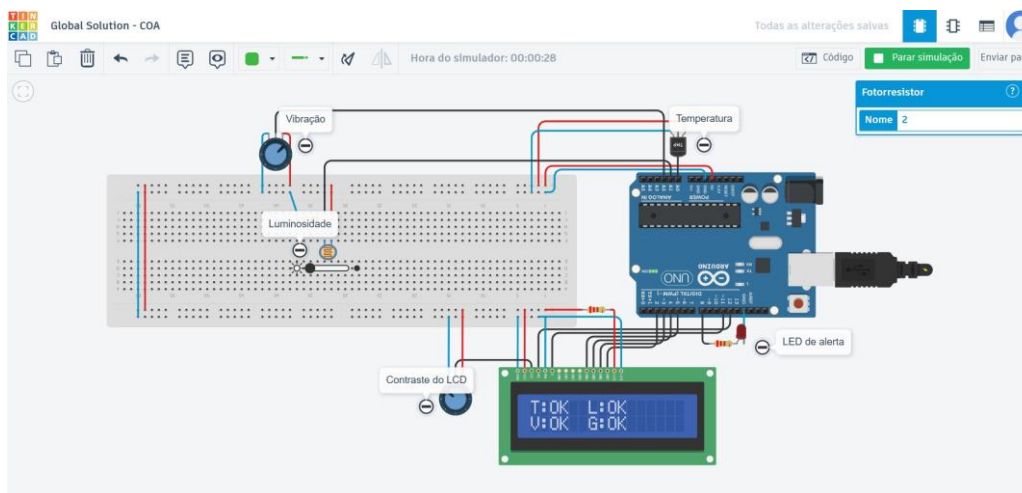


Figura 2 — Sistema em operação com todas as variáveis dentro dos limites; display exibe status OK e LED apagado.

Ao ajustar qualquer sensor ou potenciômetro para além do limite crítico configurado, o sistema detecta a condição de alerta imediatamente: o LED vermelho é acionado, a Tela 0 exibe 'G:ALRT' e a Tela 1 indica qual variável específica gerou o alerta. Este comportamento é ilustrado na Figura 3.

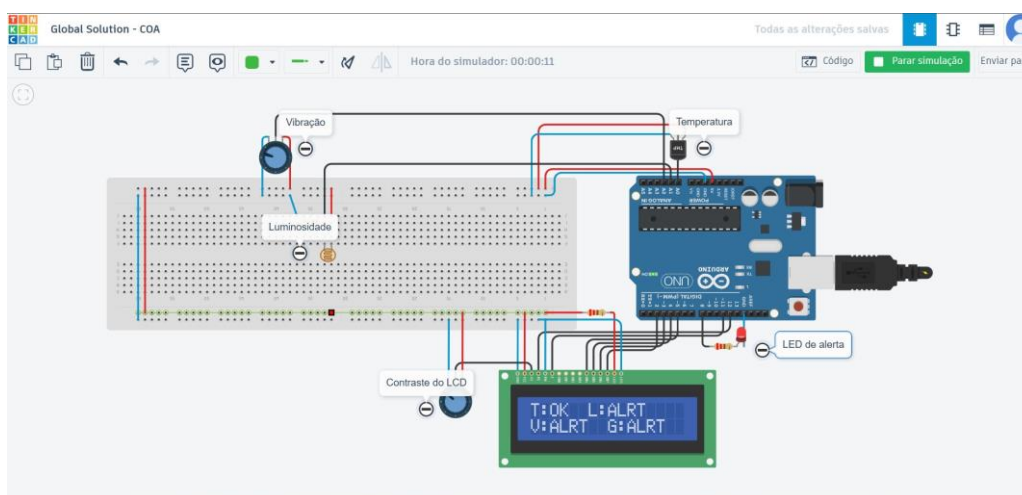


Figura 3 — Sistema em operação com condição de alerta ativa; LED aceso e display exibindo status ALRT.

Os testes confirmaram que a lógica de detecção de alertas funciona corretamente para cada variável de forma independente e que o mecanismo de alerta geral (OR logico) responde adequadamente a qualquer combinação de alertas simultâneos. A alternância de telas ocorre sem interrupção do ciclo de leitura, validando o uso de `millis()` no lugar de `delay()` para o controle de tempo.

Melhorias Futuras

Embora o sistema atenda plenamente aos requisitos propostos para a simulação, algumas melhorias podem ser consideradas em versões futuras:

- Substituição do potenciômetro por um acelerômetro real (ex.: MPU-6050) para medição efetiva de vibração nos três eixos, com cálculo de magnitude pelo módulo do vetor de aceleração;
- Implementação de comunicação sem fio (Wi-Fi via ESP32 ou LoRa) para transmissão dos dados a um servidor remoto ou plataforma de IoT (ex.: MQTT, ThingSpeak), habilitando monitoramento a distância;
- Adição de armazenamento de histórico em memória EEPROM ou cartão SD, permitindo análise retroativa das leituras e identificação de padrões de falha;
- Implementação de um buzzer ativo para alerta sonoro em complemento ao LED visual, aumentando a redundância do sistema de notificação;
- Criação de um dashboard web ou aplicativo mobile para visualização gráfica dos dados coletados em tempo real, com gráficos de histórico e limites configurados pelo usuário;
- Calibração mais precisa dos sensores com base em dados tabelados e compensação de temperatura para o LDR, aumentando a acurácia das leituras em diferentes condições ambientais.

Conclusão

Este projeto demonstrou a viabilidade de um sistema IoT embarcado para monitoramento de condições críticas em uma capsula espacial simulada, utilizando o Arduino Uno como plataforma de controle e a ferramenta Tinkercad como ambiente de simulação.

O sistema desenvolvido cumpriu todos os objetivos propostos: coleta contínua de dados dos sensores, conversão e processamento dos valores adquiridos, comparação com limites predefinidos, exibição organizada no display LCD 16x2 e acionamento do LED de alerta mediante condições críticas. A alternância não bloqueante de telas e a lógica modular de alertas individuais e geral demonstraram boas práticas de programação embarcada.

Do ponto de vista acadêmico, o projeto proporcionou experiência prática com conceitos fundamentais de sistemas embarcados, tais como: leitura analógica e conversão AD, utilização de bibliotecas de hardware, temporização não bloqueante com `millis()` e exibição formatada em displays de recursos limitados. A simulação do sensor de vibração por potenciômetro, embora seja uma simplificação, permitiu validar integralmente a lógica de detecção e resposta do sistema.

O projeto está alinhado ao ODS 9 (Inovação e Infraestrutura), contribuindo com o desenvolvimento de soluções tecnológicas robustas aplicáveis a ambientes de alta criticidade, como missões aeroespaciais.

Links do Projeto

Simulação no Tinkercad: https://www.tinkercad.com/things/7ZBku7a92Tj-global-solution-coa?sharecode=Ag-auWkleHudWy2uvOhL03sFWdusYM7DWpYb_4HR0lw

Vídeo explicativo (YouTube): <https://youtu.be/1qs2sT09RGM>

Repositório no GitHub: https://github.com/gabrielbfurin/COA-Global_Solution-2026